

Documentation technique — Mini Dashboard

Tableau de bord LVGL sur Raspberry Pi 5 (VERSION COMPLeTE)

lv_port_vscode/src | NavRelay | SETUP.sh

meka3

19 avril 2026

Table des matières

1	DÉMARRAGE RAPIDE (NOUVELLE INSTALLATION)	4
1.1	Installation en 3 etapes	4
1.2	Guide d'utilisation quotidienne	4
2	Vue d'ensemble du systeme	5
2.1	Architecture globale	5
2.2	Prerequis materiels et logiciels	5
2.2.1	Build tools (obligatoire pour compiler)	5
2.2.2	Paquets runtime et dependances	5
2.2.3	Installation automatisee (RECOMMANDÉE)	6
3	Compilation et mise en place du projet LVGL	6
3.1	Cloner le dépôt et initialiser les sous-modules	6
3.2	Structure du repertoire lv_port_vscode	7
3.3	Configuration LVGL (lv_conf.h)	7
3.4	Compilation complete avec CMake (RECOMMANDeE)	7
3.5	Compilation manuelle (alternative)	8
3.6	Lancer le dashboard	8
4	Pair Button et decouverte Bluetooth	8
4.1	Nouvelle approche simplifiee	8
4.2	Rôle de bt_pairing.h	8
4.3	Script bt_make_discoverable.sh	9
4.4	Appairage initial du telephone	9
4.4.1	Methode 1 : Via le bouton Pair du dashboard (RECOMMANDEE) . . .	9
4.4.2	Methode 2 : En ligne de commande (manuel)	9
5	Configuration systeme Bluetooth	10
5.1	Configuration BlueZ pour la musique et les appels	10
5.1.1	Desactivation du profil Headset natif de BlueZ	10
5.1.2	Activation du service oFono	10
5.1.3	Redemarrage des services Bluetooth	10
5.2	Configuration PipeWire pour HFP via oFono	10
5.3	Verification de la pile D-Bus	11

6	Daemon de liaison RFCOMM : nav-bind-daemon.sh	11
6.1	Rôle et perimetre	11
6.2	Variables de configuration	11
6.3	Logique de binding detaillee	11
6.3.1	etape 1 : lecture de l'etat courant de rfcomm1	12
6.3.2	etape 2 : consultation SDP du telephone	12
6.3.3	etape 3 : decision de rebinding	12
6.3.4	etape 4 : rebinding	12
6.4	Installation comme service systemd	12
6.5	Diagnostic et depannage	13
7	Application Android NavRelay	14
7.1	Structure du projet	14
7.2	Compiler et installer	14
7.3	Permissions et manifeste	14
7.4	MainActivity : demarrage du serveur	14
7.5	NavServer : serveur Bluetooth RFCOMM	15
7.5.1	UUID et nom de service	15
7.5.2	Cycle de vie du serveur	15
7.5.3	Envoi des messages JSON	15
7.5.4	Format JSON	15
7.6	MapsNotificationListener	16
7.6.1	Interception des notifications	16
7.6.2	Extraction de la distance	16
7.6.3	Donnees GPS	16
7.6.4	Verifier la connexion depuis le Pi	16
8	Application LVGL (lv_port_vscodc/src)	16
8.1	Macros de configuration	16
8.2	Structures de donnees partagees	16
8.3	Thread de navigation : nav_thread_fn()	17
8.3.1	Connexion et reconnexion à /dev/rfcomm1	17
8.3.2	Parsing JSON	18
8.4	Thread musique : music_thread_fn()	18
8.4.1	Connexion D-Bus et decouverte du lecteur	18
8.4.2	Lecture des proprietes	18
8.4.3	Commandes de contrôle	18
8.5	Thread appels : calls_thread_fn()	18
8.5.1	Decouverte du modem HFP	18
8.5.2	Polling des appels	18
8.5.3	Decrocher et raccrocher	18
8.6	Timer UI (100 ms)	19
8.7	Layout LVGL	19
8.8	Callbacks des boutons	19
9	Compteur de vitesse LVGL : speedo_interactive.c/.h	19
9.1	API publique	19
9.2	Parametres du cadran	20
9.3	Conversion vitesse/angle et aiguille	20

10 Recuperation de jaquette : album_art.c	20
10.1 Signature	20
10.2 Flux d'execution	20
11 Ordre de demarrage complet	21
12 Verification et validation pre-lancement	21
12.1 Checklist systeme (apres SETUP.sh)	21
12.2 Checklist appairage	21
12.3 Checklist application	21
13 Depannage general	21
13.1 « Waiting for route... » en permanence	21
13.2 « No MediaPlayer »	22
13.3 Aucun appel detecte	22
13.4 La jaquette ne s'affiche pas	22
13.5 Pair Button ne fonctionne pas	22
13.6 Commandes de logs utiles	22
14 Limitations connues	22
15 Fichiers du projet	23
15.1 Fichiers essentiels	23

1 DÉMARRAGE RAPIDE (NOUVELLE INSTALLATION)

1.1 Installation en 3 etapes

Pour une nouvelle installation de Raspberry Pi OS Bookworm, executer :

1. Cloner le dépôt et les sous-modules

```
git clone --recursive https://github.com/EEemeka33/Mini_Dashboard.git
cd Mini_Dashboard
```

Note : L'option `-recursive` remplace les deux commandes separees `git submodule update -init -recursive`, simplifiant le processus.

2. Installation et configuration automatisee (SETUP.sh)

```
bash SETUP.sh
```

Duree : 5-10 minutes. Installe paquets, configure services systemd et le daemon RF-COMM.

Le script installe et active automatiquement :

- `textttbluetooth.service` pour BlueZ
- `textttfono.service` pour les appels HFP
- `textttnav-bind.service` pour le daemon RFCOMM (nouvelle feature)
- `textttpulseaudio` pour la musique Bluetooth

3. Compiler le dashboard

```
cd lv_port_pc_vscode
rm -rf build
mkdir build
cd build
cmake ..
make -j4
```

4. Verifier que les services systemd tournent

```
sudo systemctl status bluetooth.service
sudo systemctl status ofono.service
sudo systemctl status nav-bind.service
```

Tous les trois doivent montrer `active (running)`.

1.2 Guide d'utilisation quotidienne

Consulter `QUICK_START.md` pour :

- Demarrer le dashboard apres installation
- Rendre le Pi discoverable (Pair Button)
- Appairer le telephone via l'interface Bluetooth du Pi
- Verifier que la musique, les appels et la navigation fonctionnent
- Interpretation des logs de daemon : `journalctl -u nav-bind.service -f`

2 Vue d'ensemble du systeme

Le projet Mini Dashboard est un tableau de bord automobile embarque tournant sur Raspberry Pi 5 sous un ecran circulaire 1080 × 1080 pixels. Il agrege trois flux de donnees en temps reel :

- **Navigation** : donnees de Google Maps relayees depuis un smartphone Android via Bluetooth RFCOMM (profil SPP), lues sur `/dev/rfcomm1` sous forme de lignes JSON.
- **Musique Bluetooth** : metadonnees A2DP/AVRCP et jaquette, obtenues par D-Bus via `org.bluez.MediaPlayer1`.
- **Appels HFP** : detection et gestion des appels entrants via oFono et `org.ofono.VoiceCallManager`.

2.1 Architecture globale

Smartphone Android	Couche systeme Pi	Application LVGL
NavRelay (Kotlin) Google Maps notifications	BlueZ + RFCOMM <code>/dev/rfcomm1</code>	<code>nav_thread_fn()</code> JSON → <code>nav_info_t</code>
Lecteur audio A2DP/AVRCP	BlueZ MediaPlayer1 D-Bus systeme	<code>music_thread_fn()</code> <code>get_music_properties()</code>
Telephonie Android Profil HFP	BlueZ + oFono D-Bus systeme	<code>calls_thread_fn()</code> <code>org.ofono.VoiceCall</code>

2.2 Prerequis materiels et logiciels

- Raspberry Pi 5 avec Raspberry Pi OS Bookworm (64-bit recommande)
- ecran 1080 × 1080, connecte via HDMI ou DSI
- Contrôleur Bluetooth integre ou USB (hci0)
- Smartphone Android avec l'application **NavRelay** installee et Google Maps

2.2.1 Build tools (obligatoire pour compiler)

```
sudo apt update && sudo apt install -y \
    build-essential cmake git pkg-config
```

2.2.2 Paquets runtime et dependances

```
sudo apt update
sudo apt install -y pulseaudio pulseaudio-module-bluetooth
sudo apt install -y bluez bluez-tools ofono modem-manager
sudo apt install -y libdbus-1-dev libcurl4-openssl-dev libjson-c-dev
sudo apt install -y libSDL2-dev libSDL2-image-dev
sudo apt install -y alsa-utils dbus
```

Ou en une seule commande (identique à SETUP.sh) :

```
sudo apt update
sudo apt install -y pulseaudio pulseaudio-module-bluetooth bluez
sudo apt install -y bluez-tools ofono modem-manager alsa-utils dbus
sudo apt install -y libdbus-1-dev libcurl4-openssl-dev libjson-c-dev
sudo apt install -y libSDL2-dev libSDL2-image-dev
```

2.2.3 Installation automatisée (RECOMMANDÉE)

Tous les paquets et configurations ci-dessus sont appliqués automatiquement par :

```
bash /home/pi5/dashboard/SETUP.sh
```

Ce script prend 5-10 minutes et configure également les services systemd dans les bonnes conditions.

3 Compilation et mise en place du projet LVGL

3.1 Cloner le dépôt et initialiser les sous-modules

Le projet utilise LVGL comme sous-module Git. L'option `-recursive` initialise automatiquement les sous-modules lors du clonage :

```
git clone --recursive https://github.com/EEemeka33/Mini_Dashboard.git  
cd Mini_Dashboard
```

Avantage : Une seule commande `git clone -recursive` remplace les deux commandes séparées `git clone` puis `git submodule update -init -recursive`.

3.2 Structure du repertoire lv_port_vscode

```
lv_port_vscode/  
  src/  
    hal/  
      hal.c / hal.h      # Pilote SDL2 pour LVGL  
    main.c                # Point d'entree, UI, threads  
    speedo_interactive.c  # Widget compteur de vitesse  
    speedo_interactive.h  # API publique du compteur  
    album_art.c           # Recuperation jaquette iTunes  
    bt_pairing.h          # Appels Pair button  
  lvgl/                   # Sources LVGL (sous-module)  
  lv_conf.h               # Configuration LVGL  
  CMakeLists.txt  
  build/                  # Repertoire de compilation
```

3.3 Configuration LVGL (lv_conf.h)

Les polices suivantes doivent être activées car elles sont utilisées dans `main.c` et `speedo_interactive.c` :

Listing 1 – lv_conf.h — polices à activer

```
#define LV_FONT_MONTERRAT_14 1  
#define LV_FONT_MONTERRAT_16 1  
#define LV_FONT_MONTERRAT_18 1  
#define LV_FONT_MONTERRAT_22 1  
#define LV_FONT_MONTERRAT_24 1  
#define LV_FONT_MONTERRAT_48 1
```

Le pilote système de fichiers POSIX doit être activé pour le chargement d'images JPEG :

```
#define LV_USE_FS_POSIX 1  
#define LV_FS_POSIX_LETTER 'A'
```

Le decodeur JPEG intégré doit être activé pour que `lv_image_set_src()` charge `cover.jpg` :

```
#define LV_USE_TJPGD 1 /* ou LV_USE_LIBJPEG_TURBO selon la version */
```

Ligne pour afficher l'application en Fullscreen :

```
#define LV_SDL_FULLSCREEN 1
```

3.4 Compilation complète avec CMake (RECOMMANDeE)

```
cd lv_port_vscode  
rm -rf build  
mkdir build  
cd build  
cmake ..  
make -j4
```

Ou en une seule ligne :

```
cd lv_port_vscode && rm -rf build && mkdir build && cd build && cmake ..  
&& make -j4
```

L'exécutable sera créé à `bin/main`.

3.5 Compilation manuelle (alternative)

```
cd lv_port_vscode
gcc -O2 -o dashboard \
src/main.c src/speedo_interactive.c src/album_art.c \
$(find lvgl/src -name "*.c" | tr '\n' ' ') \
hal/hal.c \
$(pkg-config --cflags --libs dbus-1 libcurl json-c sdl2) \
-I. -Ilvgl -Ilvgl/src \
-lm -lpthread
```

3.6 Lancer le dashboard

Le programme necessite une session graphique (SDL2) :

```
export DISPLAY=:0
cd lv_port_vscode/bin
./main
```

Pour un lancement automatique au demarrage, un service systemd peut être configure via SETUP.sh.

4 Pair Button et decouverte Bluetooth

4.1 Nouvelle approche simplifiee

Au lieu de tenter un appairage automatise par script (qui conflitue avec l'interface Bluetooth du Pi OS), le **Pair Button** utilise une approche deux-etapes :

1. **Utilisateur clique sur le bouton "Pair"** dans le dashboard (UI LVGL).
2. **Pi devient decouvrable** pendant 3 minutes via le script `bt_make_discoverable.sh`.
3. **Utilisateur appaire depuis Bluetooth Settings du Pi** ou du telephone.

Cela elimine entierement le conflit logiciel car une seule application (l'interface native du Pi) envoie les commandes Bluetooth.

4.2 Rôle de bt_pairing.h

Listing 2 – bt_pairing.h — fonction publique

```
1 static inline int bluetooth_pair_device_start(void)
2 {
3     /* Lance bt_make_discoverable.sh en processus fils detache */
4     pid_t pid = fork();
5     if (pid == 0) {
6         setsid(); /* Detache de la session parente */
7         execv("/bin/bash", ...);
8         exit(0);
9     }
10    return (pid > 0) ? 0 : -1;
11 }
```

La fonction retourne 0 en cas de succes, -1 en cas d'erreur. Le callback bouton affiche "Made Discoverable" ou "Error".

4.3 Script `bt_make_discoverable.sh`

Listing 3 – `bt_make_discoverable.sh`

```
#!/bin/bash
bluetoothctl discoverable on
sleep 180          # 3 minutes
bluetoothctl discoverable off
```

Ce script s'exécute en arrière-plan, sans bloquer l'UI du dashboard.

4.4 Appairage initial du téléphone

4.4.1 Méthode 1 : Via le bouton Pair du dashboard (RECOMMANDÉE)

1. Lancer le dashboard : `./bin/main` depuis `lv_port_pc_vscode/build`
2. Cliquer sur le bouton bleu « Pair » dans l'UI LVGL
3. Le Pi devient découvrable pendant 3 minutes
4. Sur le téléphone : ouvrir Paramètres → Bluetooth → Chercher des appareils
5. Sélectionner « raspberrypi » et appuyer sur Appairer
6. Confirmer le PIN sur le téléphone si demande
7. Dashboard affiche « Made Discoverable » si succès

Avantage : Aucune ligne de commande, interface graphique intégrée au dashboard. Le MAC du téléphone est automatiquement détecté par `main.c`.

4.4.2 Méthode 2 : En ligne de commande (manuel)

```
sudo bluetoothctl
[bluetooth]# power on
[bluetooth]# agent on
[bluetooth]# default-agent
[bluetooth]# scan on
```

Attendre que le téléphone apparaisse :

[NEW] Device 2C:BE:EB:7F:2D:FA Pixel 7

Continuer l'appairage :

```
[bluetooth]# scan off
[bluetooth]# pair 2C:BE:EB:7F:2D:FA
```

Confirmer le PIN sur le téléphone (ou pas si le Pi envoie le PIN automatiquement).

```
[bluetooth]# trust 2C:BE:EB:7F:2D:FA
[bluetooth]# connect 2C:BE:EB:7F:2D:FA
[bluetooth]# quit
```

Note : Après appairage, l'adresse MAC est automatiquement sauvegardée par `main.c` dans `/tmp/connected_phone_mac.txt` lors de la détection du téléphone. Aucune modification manuelle du daemon n'est nécessaire.

5 Configuration systeme Bluetooth

5.1 Configuration BlueZ pour la musique et les appels

5.1.1 Desactivation du profil Headset natif de BlueZ

Par default, BlueZ enregistre les UUID HFP/HSP pour son propre profil *Headset*, ce qui entre en conflit avec oFono. L'erreur suivante apparaît alors :

```
RegisterProfile() replied: org.bluez.Error.NotPermitted
UUID already registered
```

Pour ceder la gestion HFP à oFono, editer `/etc/bluetooth/main.conf` :

```
sudo nano /etc/bluetooth/main.conf
```

Listing 4 – `/etc/bluetooth/main.conf`

```
[General]
Disable=Headset
```

Cette directive ne desactive ni A2DP ni AVRCP ; seul le profil casque/mains libres natif BlueZ est desactive. Le lecteur de musique continue de fonctionner normalement.

5.1.2 Activation du service oFono

```
sudo systemctl enable ofono
sudo systemctl start ofono
sudo systemctl status ofono
```

5.1.3 Redemarrage des services Bluetooth

Apres toute modification de `main.conf` :

```
sudo systemctl restart bluetooth
sudo systemctl restart ofono
```

5.2 Configuration PipeWire pour HFP via oFono

Sur Raspberry Pi OS Bookworm, PipeWire remplace PulseAudio. Pour que les appels HFP passent par oFono :

```
sudo mkdir -p /etc/pipewire/pipewire-pulse.conf.d/
sudo nano /etc/pipewire/pipewire-pulse.conf.d/10-ofono.conf
```

Listing 5 – `10-ofono.conf`

```
pulse.properties = {
    bluez5.headset-roles = [ hfp_hf hsp_hs ]
    bluez5.hfp-hsp-backend = ofono
}
```

Redemarrer PipeWire dans la session utilisateur (sans `sudo`) :

```
systemctl --user restart pipewire pipewire-pulse wireplumber
```

5.3 Verification de la pile D-Bus

Apres connexion du telephone et lancement d'un morceau :

```
# Verifier la presence d'un objet MediaPlayer1 (musique)
busctl tree org.bluez
# Attendu : /org/bluez/hci0/dev_2C_BE_EB_7F_2D_FA/player0

# Inspecter les proprietes
busctl introspect org.bluez \
  /org/bluez/hci0/dev_2C_BE_EB_7F_2D_FA/player0

# Verifier oFono (appels HFP)
busctl call org.ofono / org.ofono.Manager GetModems
# Doit lister un modem avec org.ofono.VoiceCallManager
```

6 Daemon de liaison RFCOMM : nav-bind-daemon.sh

6.1 Rôle et perimetre

nav-bind-daemon.sh est le seul demon specifique au projet côte Raspberry Pi. Il maintient en permanence le binding entre /dev/rfcomm1 et le service NavJsonService de l'application Android NavRelay.

Le numero de canal RFCOMM assigne dynamiquement par Android peut varier à chaque reconnexion ; il ne peut être connu qu'en consultant le registre SDP du telephone à la volée, d'où la necessite de ce demon.

6.2 Variables de configuration

A partir de la version amelioree du daemon, le MAC du telephone est detecte automatiquement par main.c et ecrit dans le fichier /tmp/connected_phone_mac.txt.

Listing 6 – nav-bind-daemon.sh — detection auto du MAC

```
MAC_FILE="/tmp/connected_phone_mac.txt"
if [ -f "$MAC_FILE" ]; then
  PHONE_MAC=$(cat "$MAC_FILE" 2>/dev/null)
else
  PHONE_MAC="2C:BE:EB:7F:2D:FA" # Fallback si fichier absent
fi
TARGET_SERVICE_NAME="NavJsonService"
```

Avantage : Plus besoin de modifier manuellement le MAC. Le daemon se reconfigure automatiquement lors du reconnexion d'un telephone different.

Comment ca marche :

1. main.c detecte le telephone connecte via D-Bus (org.bluez.Device1).
2. main.c extrait l'adresse MAC et l'ecrit dans /tmp/connected_phone_mac.txt.
3. Le daemon relance la lecture du fichier chaque cycle (5 secondes).
4. Si le MAC change (nouveau telephone), le daemon se reconfigure automatiquement.

6.3 Logique de binding detaillee

Le script tourne dans une boucle infinie avec 5 secondes d'attente entre chaque cycle.

6.3.1 etape 1 : lecture de l'etat courant de rfcomm1

```
CURRENT_LINE=$(rfcomm | awk -v mac="$PHONE_MAC" \
  '$1=="rfcomm1:" && $2==mac {print $0}')
```

La sortie de rfcomm ressemble à :

```
rfcomm1: 2C:BE:EB:7F:2D:FA channel 7 config
```

Si aucune ligne ne correspond, CURRENT_LINE est vide et un rebind sera declenche.

6.3.2 etape 2 : consultation SDP du telephone

```
SDP_OUTPUT=$(sdptool browse "$PHONE_MAC" 2>/dev/null || true)

TARGET_CHANNEL=$(printf '%s\n' "$SDP_OUTPUT" \
  | awk -v name="$TARGET_SERVICE_NAME" '
    $0 ~ "Service Name: "name { in_block=1 }
    in_block && /Channel:/ { print $2; exit }
  ')
```

sdptool browse parcourt tous les services Bluetooth du telephone. Si TARGET_CHANNEL est vide (NavRelay non demarre ou telephone hors portee), le cycle recommence apres 5 s.

6.3.3 etape 3 : decision de rebinding

Trois cas declenchent un rebind :

- rfcomm1 n'existe pas ou n'est pas associe à PHONE_MAC.
- rfcomm1 existe mais est en etat closed.
- rfcomm1 existe mais pointe sur un canal different de TARGET_CHANNEL.

6.3.4 etape 4 : rebinding

```
rfcomm release 1 2>/dev/null || true
rfcomm bind 1 "$PHONE_MAC" "$TARGET_CHANNEL"
```

Apres un bind reussi, /dev/rfcomm1 est disponible et nav_thread_fn() peut l'ouvrir.

6.4 Installation comme service systemd

```
sudo nano /etc/systemd/system/nav-bind.service
```

Listing 7 – /etc/systemd/system/nav-bind.service

```
[Unit]
Description=RFCOMM binding daemon for NavRelay navigation
After=bluetooth.target
Wants=bluetooth.target

[Service]
Type=simple
ExecStart=/bin/bash /home/pi5/dashboard/nav-bind-daemon.sh
Restart=on-failure
RestartSec=5
StandardOutput=journal
StandardError=journal
```

```
[Install]
WantedBy=multi-user.target
```

```
# 1. Copier le script du daemon
sudo cp nav-bind-daemon.sh /usr/local/bin/nav-bind-daemon.sh

# 2. Rendre executable
sudo chmod +x /usr/local/bin/nav-bind-daemon.sh

# 3. Creer le service systemd
sudo tee /etc/systemd/system/nav-bind.service > /dev/null << 'EOF'
[Unit]
Description=RFCOMM auto-bind daemon for NavJsonService
After=bluetooth.target
Wants=bluetooth.target

[Service]
Type=simple
ExecStart=/usr/local/bin/nav-bind-daemon.sh
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
EOF

# 4. Recharger systemd
sudo systemctl daemon-reload

# 5. Activer au demarrage
sudo systemctl enable nav-bind.service

# 6. Demarrer maintenant
sudo systemctl start nav-bind.service

# 7. Verifier
systemctl status nav-bind.service
```

6.5 Diagnostic et depannage

sdptool returned nothing Le telephone n'est pas connecte. Verifier : `bluetoothctl info 2C:BE:EB:7F:2D:FA` (champ Connected: yes).

cannot find service "NavJsonService" NavRelay n'est pas demarre. Ouvrir NavRelay et appuyer sur « Start ».

rfcomm bind failed Module noyau manquant : `sudo modprobe rfcomm`. Verifier aussi que le service tourne en root.

/dev/rfcomm1 introuvable apres binding Ajouter l'utilisateur au groupe dialout : `sudo usermod -aG dialout pi5`, puis se reconnecter.

rfcomm1 se rebind en boucle Qualite de signal Bluetooth insuffisante ou telephone non marque comme de confiance (`trust` dans `bluetoothctl`).

7 Application Android NavRelay

7.1 Structure du projet

```
NavRelay/
  app/src/main/
    AndroidManifest.xml
    java/com/example/navrelay/
      MainActivity.kt
      NavServer.kt
      MapsNotificationListener.kt
    res/layout/
      activity_main.xml
```

7.2 Compiler et installer

1. Ouvrir Android Studio et importer le projet NavRelay/.
2. Verifier que `compileSdkVersion ≥ 33`.
3. Connecter le telephone en USB avec le debogage active.
4. Cliquer sur *Run* ou : `./gradlew installDebug`

7.3 Permissions et manifeste

- BLUETOOTH, BLUETOOTH_ADMIN, BLUETOOTH_CONNECT, BLUETOOTH_ADVERTISE, BLUETOOTH_SCAN
- ACCESS_FINE_LOCATION, ACCESS_COARSE_LOCATION
- POST_NOTIFICATIONS (Android 13+)
- Service MapsNotificationListener declare avec l'action BIND_NOTIFICATION_LISTENER_SERVICE

IMPORTANT — **etape obligatoire apres installation** : Accorder l'accès aux notifications manuellement apres le premier lancement :

1. Ouvrir *Parametres* → *Applications*
2. Chercher *Acces speciaux* ou *Permissions avancees*
3. Selectionner *Acces aux notifications*
4. Activer NavRelay

MainActivity ouvre automatiquement cet ecran via `Settings.ACTION_NOTIFICATION_LISTENER_SETTINGS`.

Note : Sur certaines ROM Android, desactiver l'optimisation de batterie pour NavRelay afin d'éviter que le service soit tue en arriere-plan.

7.4 MainActivity : demarrage du serveur

Listing 8 – MainActivity.kt — bouton Start

```
1 startButton.setOnClickListener {
2     NavServer.start(this)
3     textStatus.text = "NavServer␣demarre.\n" +
4         "Active␣le␣service␣de␣notifications␣pour␣NavRelay."
5     startActivity(
6         Intent(Settings.ACTION_NOTIFICATION_LISTENER_SETTINGS))
7 }
```

Les permissions Bluetooth et localisation sont demandees au lancement via `ActivityCompat.requestPermissions` avec les variantes `BLUETOOTH_CONNECT` / `ADVERTISE` / `SCAN` pour Android 12+.

7.5 NavServer : serveur Bluetooth RFCOMM

7.5.1 UUID et nom de service

Listing 9 – NavServer.kt — constantes

```
1 private const val SERVICE_NAME = "NavJsonService"
2 val SPP_UUID: UUID =
3     UUID.fromString("00001101-0000-1000-8000-00805F9B34FB")
```

L'UUID 00001101-... est l'UUID standardise du *Serial Port Profile* (SPP). `NavJsonService` est le nom que `nav-bind-daemon.sh` recherche dans le registre SDP. Ces deux valeurs doivent rester coherentes.

7.5.2 Cycle de vie du serveur

`start()` lance un thread qui :

1. Recupere l'adaptateur Bluetooth et verifie qu'il est actif.
2. Appelle `listenUsingRfcommWithServiceRecord(SERVICE_NAME, SPP_UUID)`, enregistrant le service dans le registre SDP avec un canal assigne dynamiquement. C'est ce canal que le daemon decouvre via `sdptool`.
3. Boucle sur `accept()` pour accepter la connexion du Raspberry Pi. À chaque reconnexion, l'ancienne socket et son flux sont fermes et remplaces.

7.5.3 Envoi des messages JSON

Listing 10 – NavServer.kt — `send()`

```
1 fun send(json: JSONObject) {
2     val out = outRef.get() ?: return
3     try {
4         val line = json.toString() + "\n"
5         out.write(line.toByteArray(Charsets.UTF_8))
6         out.flush()
7     } catch (e: Exception) {
8         e.printStackTrace()
9         stop()
10    }
11 }
```

Chaque message est une ligne JSON terminee par `\n` en UTF-8. En cas d'exception d'ecriture (deconnexion Bluetooth), le serveur s'arrête proprement et attend une nouvelle connexion.

7.5.4 Format JSON

Listing 11 – Exemple de message JSON envoye par NavRelay

```
{
  "timestamp": 1711550400,
  "next_action": "Tournez a droite sur Rue de la Paix",
  "distance_to_next_m": 320,
  "speed_kmh": 48.5,
  "lat": 48.8566,
  "lon": 2.3522
}
```

7.6 MapsNotificationListener

7.6.1 Interception des notifications

`onNotificationPosted()` filtre les notifications dont le `packageName` est `"com.google.android.apps.maps"` et extrait :

- `EXTRA_TITLE` : distance jusqu'à la prochaine manœuvre (ex : « 320 m »).
- `EXTRA_TEXT` : instruction textuelle (ex : « Tournez à droite »).
- `EXTRA_SUB_TEXT` : information complémentaire.

Le champ `next_action` est determine par ordre de priorite : `EXTRA_TEXT > EXTRA_SUB_TEXT > EXTRA_TITLE`.

7.6.2 Extraction de la distance

`parseDistanceMeters()` analyse la chaîne concatenee avec une expression reguliere reconnaissant les formats « 320 m » et « 1.2 km ». Les kilometres sont convertis en metres pour normaliser `distance_to_next_m`.

7.6.3 Donnees GPS

`requestLocationUpdates()` demande des mises à jour GPS toutes les secondes (deplacement minimal 1 m). La vitesse `Location.speed` (m/s) est convertie en km/h (`speed * 3.6f`) dans `onLocationChanged()`, puis integree dans le prochain message JSON.

7.6.4 Verifier la connexion depuis le Pi

```
# Verifier que NavJsonService est visible en SDP
sdptool browse 2C:BE:EB:7F:2D:FA | grep -A5 "NavJsonService"
# Sortie attendue :
#   Service Name: NavJsonService
#   Channel: 7      (numero variable)

# Lire le flux JSON en direct
cat /dev/rfcomm1
```

8 Application LVGL (lv_port_vscode/src)

8.1 Macros de configuration

Listing 12 – main.c — macros de configuration

```
1 #define SCREEN_W          1080
2 #define SCREEN_H          1080
3 #define MUSIC_COVER_PATH_FS  "/home/pi5/dashboard/cover.jpg"
4 #define MUSIC_COVER_PATH    "A:/home/pi5/dashboard/cover.jpg"
5 #define CENTER_RADIUS      250
6 #define BOTTOM_PLAYER_HEIGHT 250
```

`MUSIC_COVER_PATH_FS` est le chemin POSIX reel pour ecrire et verifier la jaquette. `MUSIC_COVER_PATH` utilise le prefixe `A:` du pilote LVGL POSIX initialise par `lv_fs_posix_init()`.

8.2 Structures de donnees partagees

Toutes les donnees partagees entre les threads et le timer UI sont stockees dans l'instance globale `g_state` de type `app_state_t`, protegee par `g_state.lock` (mutex POSIX).

Listing 13 – main.c — structures de donnees

```

1 typedef struct {
2     char    instruction[256]; /* Instruction de virage courante */
3     double  distance;        /* Distance en metres */
4     double  next_distance;
5     char    street_name[256]; /* Nom de la rue courante */
6     double  speed;           /* Vitesse vehicule en km/h */
7     double  lat;
8     double  lon;
9 } nav_info_t;
10
11 typedef struct {
12     char    title[256];
13     char    artist[256];
14     char    album[256];
15     char    status[32];      /* "playing", "paused", etc. */
16     unsigned int position_ms;
17     unsigned int duration_ms;
18     char    cover_path[512];
19 } music_info_t;
20
21 typedef struct {
22     char    state[32];       /* "active","incoming","dialing"... */
23     char    line_id[64];     /* Numero de telephone */
24     char    name[64];        /* Nom du contact */
25     char    call_path[256];  /* Chemin D-Bus de l'appel oFono */
26 } call_info_t;
27
28 typedef struct {
29     nav_info_t    nav;
30     music_info_t  music;
31     call_info_t   current_call;
32     int           has_incoming_call;
33     pthread_mutex_t lock;
34 } app_state_t;

```

8.3 Thread de navigation : nav_thread_fn()

8.3.1 Connexion et reconnexion à /dev/rfcomm1

Listing 14 – main.c — boucle de reconnexion rfcomm1

```

1 while (1) {
2     if (fd < 0) {
3         fd = open("/dev/rfcomm1", O_RDONLY | O_NOCTTY);
4         if (fd < 0) { sleep(2); continue; }
5         f = fdopen(fd, "r");
6         if (!f) { close(fd); fd = -1; sleep(2); continue; }
7     }
8     if (fgets(line, sizeof(line), f)) {
9         /* strip \r\n, parse JSON, mise a jour g_state.nav */
10    } else {
11        /* EOF ou erreur : fermeture et reconnexion */
12        fclose(f); f = NULL; fd = -1; sleep(2);
13    }
14 }

```

Le port est ouvert avec `O_NOCTTY` pour eviter d'en faire le terminal de contrôle du processus.

8.3.2 Parsing JSON

`parse_nav_json()` utilise `json_tokenizer_parse()` de `json-c`. Elle extrait `next_action`, `distance_to_next_street_name`, `speed_kmh`, `lat`, `lon` et remplit `nav_info_t`. Si `street_name` est absent, l'UI affiche « Unknown Street ».

8.4 Thread musique : `music_thread_fn()`

8.4.1 Connexion D-Bus et decouverte du lecteur

`dbus_connect_system()` initialise une connexion au bus D-Bus systeme via `dbus_bus_get(DBUS_BUS_SYSTEM, &err)`.

`find_player_path()` envoie `GetManagedObjects` à `org.bluez` sur `/` et cherche le premier objet exposant `org.bluez.MediaPlayer1`. Le chemin resultant est de la forme :

```
/org/bluez/hci0/dev_2C_BE_EB_7F_2D_FA/player0
```

8.4.2 Lecture des proprietes

Toutes les 2 secondes, `get_music_properties()` appelle `GetAll("org.bluez.MediaPlayer1")` et extrait :

- `Status` (chaîne) → `g_state.music.status`
- `Position` (uint32, ms) → `g_state.music.position_ms`
- `Track` (dict) → `Title`, `Artist`, `Album`, `Duration`

En cas de changement de piste (couple titre+artiste), `download_itunes_cover()` est appelee pour recuperer la nouvelle jaquette.

8.4.3 Commandes de contrôle

`send_player_method(method)` envoie `Play`, `Pause`, `Next` ou `Previous` à `org.bluez.MediaPlayer1` avec un timeout de 5000 ms. Ces appels sont declenches par les boutons LVGL sous le verrou `g_state.lock`.

8.5 Thread appels : `calls_thread_fn()`

8.5.1 Decouverte du modem HFP

`find_modem_with_voicecall()` interroge `org.ofono.Manager.GetModems()` et selectionne le modem dont la liste d'interfaces contient `"org.ofono.VoiceCallManager"` :

```
/hfp/org/bluez/hci0/dev_2C_BE_EB_7F_2D_FA
```

8.5.2 Polling des appels

Toutes les 2 secondes, `get_calls()` envoie `VoiceCallManager.GetCalls()` et extrait pour chaque appel `State`, `LineIdentification` et `Name`. Le thread met à jour `has_incoming_call` si un appel est en etat `"incoming"` ou `"waiting"`.

8.5.3 Decrocher et raccrocher

`call_voicecall_method(call_path, method)` envoie `Answer` ou `Hangup` à `org.ofono.VoiceCall` (timeout 5 s).

8.6 Timer UI (100 ms)

Listing 15 – main.c — callback du timer LVGL

```
1 static void ui_update_timer(lv_timer_t *timer) {
2     update_nav_display();
3     update_music_display();
4     update_call_display();
5     speedo_set_speed((int32_t)g_state.nav.speed);
6 }
```

`update_nav_display()` Instruction, distance (m ou km), nom de rue ou « Unknown Street ».

`update_music_display()` Titre, artiste, jaquette rechargée via `stat() + lv_image_set_src()`, label Play/Pause.

`update_call_display()` Affiche/masque le panneau (`LV_OBJ_FLAG_HIDDEN`), nom ou « Unknown » et numéro de téléphone.

`speedo_set_speed()` Compteur mis à jour avec `g_state.nav.speed` en km/h entier.

8.7 Layout LVGL

Widget	Taille	Position	Fond
Compteur (speedo)	1080 × 1080	Plein écran	transparent
Conteneur nav.	500 × 500	Centre, +100 px Y	#1a1a2e
Conteneur musique	830 × 220	Bas, −150 px Y	#0f0f1e
Jaquette album	100 × 100	Gauche musique	—
Panneau appel	600 × 250	Par-dessus musique	#393a4f
Pair Button	Petit	Sous musique	Bleu

Le panneau d'appel est créé avec `LV_OBJ_FLAG_HIDDEN` et n'est rendu visible que lors d'un appel entrant. Le Pair Button appelle `bluetooth_pair_device_start()`.

8.8 Callbacks des boutons

Six boutons LVGL sont définis :

Bouton	Fonction	Interface
Pair	<code>bluetooth_pair_device_start()</code>	<code>bt_make_discoverable.sh</code>
Précédent	<code>Previous</code>	<code>org.bluez.MediaPlayer1</code>
Play/Pause	<code>Play / Pause</code>	<code>org.bluez.MediaPlayer1</code>
Suivant	<code>Next</code>	<code>org.bluez.MediaPlayer1</code>
Décrocher	<code>Answer</code>	<code>org.ofono.VoiceCall</code>
Raccrocher	<code>Hangup</code>	<code>org.ofono.VoiceCall</code>

9 Compteur de vitesse LVGL : speedo_interactive.c/.h

9.1 API publique

Listing 16 – speedo_interactive.h

```
1 void speedo_interactive_create(void);
2 void speedo_set_speed(int32_t speed);
```

`speedo_interactive_create()` doit être appelée une seule fois dans `create_ui()`. `speedo_set_speed()` est appelée depuis le timer UI à chaque tick de 100 ms.

9.2 Parametres du cadran

Listing 17 – speedo_interactive.c — constantes

```
1 #define SPEEDO_W      1080    /* Largeur totale (px)          */
2 #define SPEEDO_H      1080    /* Hauteur totale (px)         */
3 #define SPEED_MIN     0       /* Vitesse minimale (km/h)     */
4 #define SPEED_MAX     260     /* Vitesse maximale (km/h)     */
5 #define ARC_SWEEP     180     /* Balayage angulaire total (degres) */
6 #define ARC_START     270     /* Angle de depart (bas du cercle) */
```

9.3 Conversion vitesse/angle et aiguille

speed_to_angle() calcule l'angle par interpolation lineaire :

$$\theta = \text{ARC_START} + \frac{v - v_{\min}}{v_{\max} - v_{\min}} \times \text{ARC_SWEEP}$$

update_needle() convertit θ en radians, calcule les deux extremités de l'aiguille sur la couronne externe du cadran et met à jour l'objet ligne LVGL `needle_line`.

create_ticks_and_labels() cree pour chaque pas de 5 km/h :

- Graduation majeure tous les 10 km/h (plus longue, couleur claire).
- Graduation mineure tous les 5 km/h.
- Label numerique (police `lv_font_montserrat_18`) tous les 25 km/h.

speedo_interactive_create() cree les graduations, l'aiguille, et deux labels centraux : la valeur numerique (police 48 pt) et l'unite « km/h ».

10 Recuperation de jaquette : album_art.c

10.1 Signature

```
1 int download_itunes_cover(const char *artist,
2                           const char *album,
3                           const char *out_path);
```

Dependances : libcurl et json-c. Retourne 1 en succes, 0 sinon.

10.2 Flux d'execution

1. Encodage URL du terme (*artiste* + espace + *album*) via `curl_easy_escape()`.
2. Requête GET vers : `https://itunes.apple.com/search?term=<query>&entity=album&limit=1`
3. Accumulation de la reponse JSON en memoire via `write_to_memory()` (structure `Memory` avec `realloc()` dynamique).
4. Extraction de "artworkUrl100" dans `results[0]`.
5. Telechargement de l'image vers `out_path` via `write_to_file()`.

Si le telechargement echoue (pas de reseau, artiste inconnu), la fonction retourne 0 et l'ancienne jaquette reste affichee jusqu'au prochain changement de piste.

11 Ordre de demarrage complet

1. **Demarrer le Raspberry Pi** : bluetooth, ofono et nav-bind démarrent automatiquement via systemd (apres SETUP.sh).
2. **Sur le telephone** : activer le Bluetooth.
3. **Lancer le dashboard** : `./bin/main` depuis `lv_port_vscode/build`.
4. **Cliquer sur le bouton Pair** dans le dashboard.
5. **Appairer le telephone** via Parametres Bluetooth du Pi.
6. **Ouvrir NavRelay** et appuyer sur **Start**.
7. **Lancer Google Maps** et une navigation.

Thread	Source de donnees	Intervalle
nav_thread_fn	/dev/rfcomm1 (JSON)	Bloquant sur fgets()
music_thread_fn	D-Bus MediaPlayer1	2 secondes
calls_thread_fn	D-Bus VoiceCallManager	2 secondes
Timer UI	lv_timer_handler()	100 ms

12 Verification et validation pre-lancement

12.1 Checklist systeme (apres SETUP.sh)

Bluetooth démarre : `sudo systemctl status bluetooth`
oFono démarre : `sudo systemctl status ofono`
nav-bind service actif : `sudo systemctl status nav-bind.service`
Adaptateur visible : `hciconfig` (affiche hci0)

12.2 Checklist appairage

Telephone detectable : `bluetoothctl scan on`
Telephone appaire : `bluetoothctl paired-devices`
Telephone de confiance : `bluetoothctl trust <MAC>`
Telephone connecte : `bluetoothctl info <MAC>` → Connected: yes

12.3 Checklist application

Executable compile : `ls -la lv_port_vscode/bin/main`
Affichage X11 disponible : `echo $DISPLAY` → :0
Permissions dialout : `groups pi5` → contient dialout

13 Depannage general

13.1 « Waiting for route... » en permanence

- Verifier que `/dev/rfcomm1` existe : `ls /dev/rfcomm*`
- Tester la reception : `cat /dev/rfcomm1` (doit afficher des lignes JSON).
- Verifier les permissions : `sudo usermod -aG dialout pi5`.
- Dans NavRelay : cliquer sur **Start**, verifier que l'etat passe à "NavServer running".

13.2 « No MediaPlayer »

- Verifier A2DP avec `busctl tree org.bluez` (chercher `/player0`).
- Jouer un morceau sur le telephone pour que BlueZ cree `MediaPlayer1`.
- Redemarrer le dashboard si le player est apparu apres son demarrage.

13.3 Aucun appel detecte

- Verifier `oFono` : `systemctl status ofono`.
- Verifier `Disable=Headset` dans `/etc/bluetooth/main.conf`.
- Verifier l'existence de `/etc/pipewire/pipewire-pulse.conf.d/10-ofono.conf`.
- Tester : `busctl call org.ofono / org.ofono.Manager GetModems`.

13.4 La jaquette ne s'affiche pas

- Tester la connectivite : `curl "https://itunes.apple.com/search?term=test&entity=album&limit=1"`
- Verifier que `/home/pi5/dashboard/` est accessible en ecriture.
- S'assurer que `libcurl` et `libjson-c` sont lies à la compilation.

13.5 Pair Button ne fonctionne pas

- Verifier que `bt_make_discoverable.sh` existe et est executable : `ls -la /home/pi5/dashboard/bt_make_discoverable.sh`
- Tester manuellement : `bash /home/pi5/dashboard/bt_make_discoverable.sh`
- Verifier les logs dashboard : `journalctl -fu dashboard.service`

13.6 Commandes de logs utiles

```
journalctl -fu nav-bind.service           # Daemon RFCOMM
journalctl -fu dashboard.service          # Application LVGL
journalctl -fu bluetooth.service          # BlueZ
journalctl -fu ofono.service              # oFono
dbus-monitor --system \
    "type=signal,sender=org.bluez"         # Evenements BlueZ
dbus-monitor --system \
    "type=signal,sender=org.ofono"         # Evenements oFono
```

14 Limitations connues

Nom de l'appelant (HFP) La propriete `Name` de `org.ofono.VoiceCall` est generalement vide avec les telephones Android modernes. Seul le numero (`LineIdentification`) est affiche de maniere fiable.

Jaquette Bluetooth (AVRCP Cover Art) BlueZ ne supporte pas AVRCP 1.6 de maniere stable. La jaquette est recuperee via l'API iTunes Search, ce qui necessite une connexion Internet et introduit un delai au changement de piste.

Redecouverte du player BlueZ `find_player_path()` est appelee une seule fois au demarrage du thread musique. En cas de deconnexion Bluetooth pendant l'execution, il faut redemarrer le dashboard pour redecouvrir le player.

Canal RFCOMM variable Le canal assigne à `NavJsonService` par Android peut changer à chaque redemarrage. Ce cas est gere automatiquement par `nav-bind-daemon.sh` via le registre SDP.

Approach systeme vs. script La nouvelle approche du Pair Button utilise l'interface Bluetooth native du Pi OS plutôt qu'une tentative automatisee, ce qui elimine les conflits logiciels.

15 Fichiers du projet

15.1 Fichiers essentiels

`SETUP.sh` Script d'installation une fois (4 KB). À exécuter après la première installation, installe tous les paquets et services systemd.

`QUICK_START.md` Guide d'utilisation pour l'utilisateur final. À consulter pour le démarrage quotidien et les procédures courantes.

`nav-bind-daemon.sh` Daemon de liaison RFCOMM. Maintient le binding entre `/dev/rfcomm1` et le service NavRelay du téléphone. À adapter pour un autre MAC.

`bt_make_discoverable.sh` Rend le Pi découvrable pendant 3 minutes. Appelé par le Pair Button.

`bt_setup_audio.sh` Configuration optionnelle de PulseAudio pour Bluetooth. Généralement non nécessaire après `SETUP.sh`.

`cover.jpg` Cache pour la jaquette de l'album courant (vide initialement).